

Companion Enterprise

Architecture and action plan

■ 6 October 2026 ■ Working title ■ Built on the Local LLM PC Companion

We will turn the Local LLM PC Companion, today a one-person app on one PC, into a shared company service. People use a desktop app, signed in with their company account. A server, which can run on one machine or across many, hosts our own open-weight models and can also pass requests to OpenAI, Anthropic or Google. It answers from company documents with citations (RAG), connects to Jira and Outlook, gives each team only the tools it is allowed, and records who asked what in an audit log. A separate dashboard lets the team running it manage models, users, tools, rules and costs.

For people using it

- Install an app, sign in with the company account
- Pick from the models the company approved
- Get answers from company documents, with the sources named (RAG)
- Switch on the plugins their team is allowed
- Connect their own Jira and Outlook
- The coding agent works on files on their own PC

For the team running it

- One dashboard: models, providers, users and groups, tools, documents, rules, audit log, usage
- Settings set and locked centrally
- Add a model or a provider without touching the apps

For the company

- With our own models, nothing leaves the building
- Outside providers only through our rules
- A complete, tamper-evident audit trail
- Costs visible per person and team
- Grows by adding servers

01 Decisions so far

Topic	Decision	Status
Starting point	A copy of the Local LLM PC Companion, including its latest tool fixes, becomes the new project.	Decided
Coding agent	The agent's thinking runs on the server; reading and writing files and running commands happen on the user's PC, through the desktop app.	Decided
Where it runs	Our own servers or AWS. The design covers both, plus a one-machine version.	Open
Running it on a laptop	Install the same server on the laptop and point the app at it. No separate offline mode inside the app.	Decided
Document search (RAG)	Answers from company documents with citations, searched by meaning and keywords; access rights applied at search time; collections can be kept away from outside providers.	Decided
Plug-in tools (MCP)	Tools are MCP servers that admins add or remove and give to the teams that need them; users switch on what they are allowed.	Decided
Email	Outlook first; Gmail later.	Decided
Jira	Jira Cloud.	Decided

Topic	Decision	Status
Company sign-in	Microsoft Entra ID (the company's Microsoft 365 accounts), following from Outlook.	To confirm

02 Where we start

The Companion is one program on one PC: a Rust backend that does the work, React screens in the browser, a llama.cpp model server it starts and tunes itself, and one database file. About 34,000 lines of backend code (48,000 with tests), 12,000 lines of screens, and more than 600 automated tests.

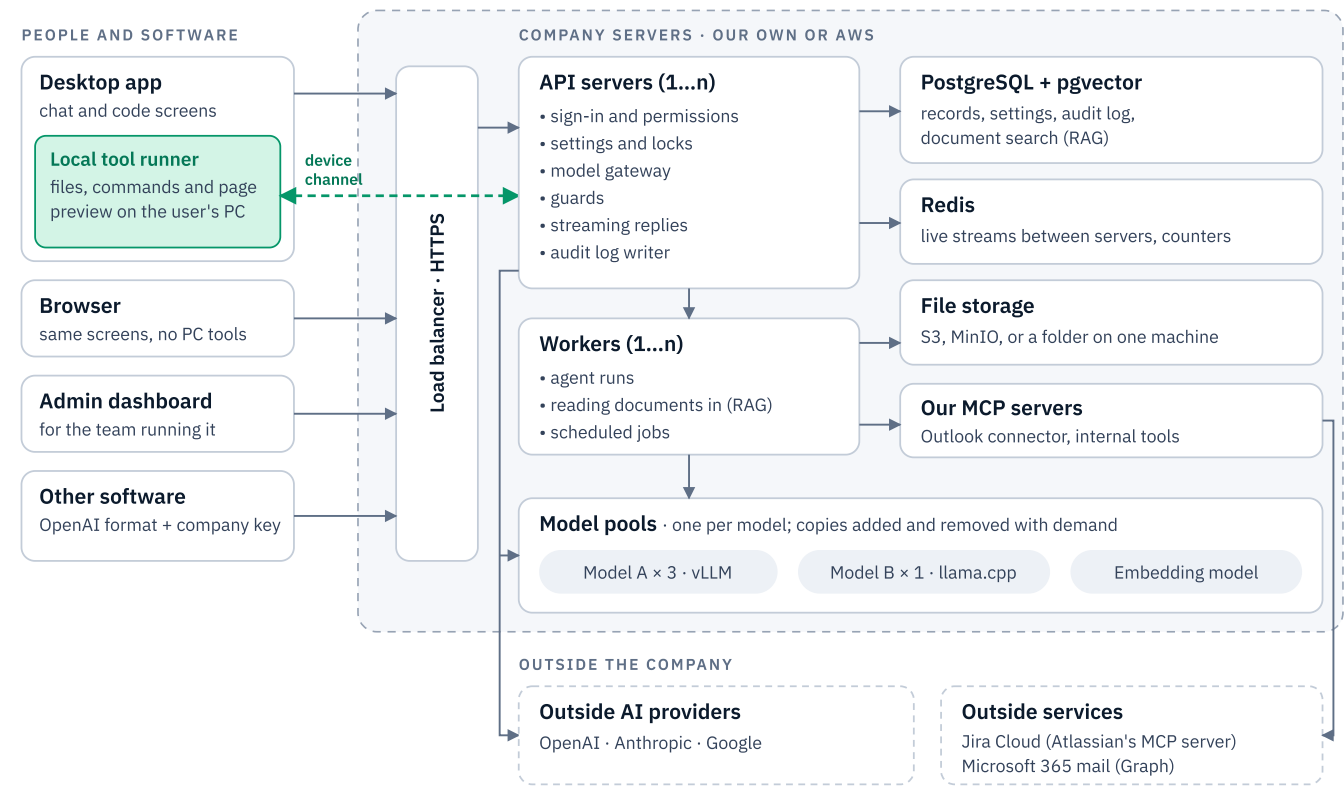
What we keep

- The coding agent: reads, writes, runs and checks project files, and stops itself when it is stuck in a loop
- Permission modes: ask, accept edits, plan, auto
- The test that checks whether a model can really call tools and write files intact
- Memory management for long tasks
- llama.cpp tuning: memory fit and per-model calibration
- Full logs on disk, saved as a zip to share

What has to change

- Nothing knows who is asking: there is one user
- One loaded model at a time
- One database file on one PC
- One settings file shared by everything
- The agent's tools assume the files are on the same machine as the server
- It only accepts connections from the same PC

03 Architecture at a glance



The green dashed line is the **device channel**: when an agent run needs to read a file or run a command, the server sends the request down this live connection, and the desktop app does it on the user's PC, inside folders the user allowed, then sends the result back. Dashed boxes are outside the company.

The pieces

Piece	What it does	Built from
Desktop app	Chat and code screens; saved servers, connection test, sign-in; runs file and command tools on the user's PC	Today's React screens in Tauri; the Companion's file, command and preview tools move into a shared library the app uses
Admin dashboard	Models, providers, users and groups, settings and locks, plugins, documents, rules, audit log, usage	New React web app sharing the same UI parts
API servers	Sign-in, permission checks, settings, model gateway, guards, streaming replies, audit records. Keep nothing in their own memory, so copies can be added freely	The Companion's backend, reworked
Workers	Agent runs, reading documents into the search index (RAG), scheduled jobs	The same program in a second role: one program to build and ship
Model pools	Each model runs as a group of copies; copies are added or removed with demand	llama.cpp (any machine, even without a GPU) or vLLM (many users on large GPU servers)
Shared storage	Everything that must last, document search, live streams between servers, files	PostgreSQL + pgvector, Redis (only with 2+ servers), S3 / MinIO / a folder

Simplified on purpose. One company per installation (serving several companies from one installation is not planned). One backend program with two roles instead of many small services. Background jobs are queued in PostgreSQL itself, with no separate queue product until measurements say otherwise.

04 How a request travels

A chat message

- 1 The app sends the message with the person's sign-in token. The server knows who they are, their groups and their settings.
- 2 Checks: is this model allowed for them, and is there budget left?
- 3 If document search is on, it finds passages, only from documents this person may open.
- 4 It builds the request with the conversation's start kept unchanged, so the model does not re-read it (the Companion's speed trick).
- 5 If the model is outside the company, the request is scanned for secrets and personal data, and collections marked "never send outside" are left out.
- 6 The same conversation always goes to the same copy of a local model, so that copy reuses what it already read.
- 7 The reply streams back; if the person is connected to another server, Redis carries it across.
- 8 An audit record and the usage count are written.

A coding-agent task

- 1 The user starts a task in the desktop app on a folder of their PC.
- 2 A worker takes the task and runs the Companion's agent loop: ask the model, carry out its next step, repeat.
- 3 Steps that touch the PC (read or write a file, run a command, preview a page) go to the app over the device channel. The app checks the folder is allowed, asks the user when the mode requires it, does the work and returns the result.
- 4 Steps that live on the server (document search, Jira, Outlook, web search) run in the worker.
- 5 The host's own looks at the project (project summary, instructions file, git status, notes) also go through the app.
- 6 If the app disconnects, the task pauses and waits instead of failing.
- 7 Every step is recorded on the server.

05 People, sign-in and settings

People sign in with their company account through Microsoft Entra ID, so the server never stores passwords, and groups come from the company directory, so joining or leaving a team takes effect without extra work. Other software uses API keys issued in the dashboard.

Role	Can do
User	Chat, run tasks, use approved models and plugins, connect their own accounts, see their own history
Team admin	Manage their team's documents, plugins and limits within what the platform admin allows
Platform admin	Everything in the dashboard: models, providers, servers, rules, company-wide settings
Auditor	Read and export the audit log; cannot change anything

Settings have three levels: a company default, an optional override for a group, and the user's own choice. Any setting can be locked by the admin; a locked setting shows as locked in the app with the reason, and the server enforces it as well.

Stays with the user (unless locked)	Moves to the admin dashboard
Theme, density, reduce motion, shortcuts, language	Performance and tuning: threads, GPU layers, cache precision, batch sizes, speculative decoding, context size
Which model to use, from the allowed list	Which models exist, downloads, calibration
Reasoning on or off	Web search provider and its keys, network policy
Permission mode, up to the admin's maximum	Allowed and blocked folders, file size limits, command time limit
Which allowed plugins are switched on	What is recorded, log level, how long records are kept
Their own Jira and Outlook connections, saved servers	Memory management defaults for long tasks

06 Models: our own and outside providers

- **One model list.** Each entry is either a local model or a provider's model, with: engine, the smallest and largest number of copies, which groups may use it, and its abilities.
- **Two engines for our own models.** llama.cpp runs anywhere, including laptops and machines without a GPU, and keeps the Companion's tuning. vLLM serves many people at once from large GPU servers (Linux).
- **Outside providers through two small adapters.** One speaks the OpenAI format, which covers OpenAI, Azure OpenAI, vLLM, llama.cpp and Google Gemini's compatible endpoint; one speaks Anthropic's own format, which handles tools better for Claude.
- **Linking a provider takes four steps in the dashboard:** add it (type, address, key), test the connection, pick which of its models to offer, and give them company names. A name such as "company-default" can point at any model and be moved later without touching the apps.
- **Abilities are measured, not assumed.** Today the Companion tests each model once: can it call a tool, and can it write a file intact? Every model gets this test, outside ones included, and the result is recorded with its context size, vision and reasoning.
- **Limits.** Tokens and money per person, team and model; usage shown in the dashboard.

07 Tools, plugins and MCP

MCP is an open standard for connecting AI apps to tools. An MCP server is a small program or web address that offers a set of tools, such as "search Jira issues" or "draft an email". Many services now publish one.

- **One plugin format.** A plugin is an MCP server plus a short description: its name, each tool's risk (reads, writes, or dangerous), what it needs from the model, and which account it needs. The Companion's own file and command tools

are described the same way.

- **Three levels of control.** The admin's catalogue decides which groups get each plugin, which actions always need approval, and whether it may be used with outside models. Users switch on what they are allowed. Each request offers the model only tools that are switched on, allowed, and usable by that model.
- **Can this model use it?** Each plugin states what it needs (tool calling, writing, a minimum context, vision). A plugin the chosen model cannot use is greyed out with the reason, and the server refuses it anyway. Models with no tool support get chat and reading only.
- **Adding one.** Remote MCP servers are added by address. MCP servers that run as programs are installed by admins only.

08 Jira and Outlook

Service	How we connect
Jira Cloud	Atlassian's official MCP server (generally available since February 2026). It respects each person's own Jira permissions.
Outlook	Our own small MCP server over Microsoft Graph, Microsoft's standard interface to mail and calendar. Microsoft's official mail server is a preview that needs a Microsoft 365 Copilot licence. Read, search and draft first; sending only after the user approves.
Gmail (later)	Google's official Gmail MCP server is a developer preview that reads and drafts but cannot send; reviewed when Gmail is scheduled.

Each person connects their own account through the usual "Sign in with Microsoft / Atlassian" screen, so the model can only see and do what that person can; their sign-in tokens are stored encrypted on the server. Sending an email or changing a ticket always asks first, whatever mode the user is in, because an email is the easiest way to slip instructions to a model; text from emails is treated as information, never as orders.

09 Document search (RAG)

RAG hands the model the right passages from company documents at the moment a question is asked, so it answers from them and cites them. Today the Companion searches one project's files by keyword.

- **Collections:** company-wide, team and personal, each with group access.
- **Reading in:** upload or connect a source; a worker pulls the text out, splits it into passages and gives each a "meaning fingerprint" (an embedding) using a local model.
- **Searching:** by meaning and keywords together, best matches first, with citations in the answer.
- **Access rights at search time:** a person only gets passages from documents they may open.
- **Labels:** a collection marked "never send outside" is only used with our own models; an "answer only from sources, or say you don't know" mode per assistant.
- **Sources:** uploads and shared folders first; Jira and other company systems later, through the same connectors as the tools.

10 Guards and the audit log

Guard	What it does
Identity	Every request carries who is asking.
Access	Document search only returns what that person may open.
Tools	An allowed list per group; write actions need approval; the admin can make some approvals mandatory in every mode; the app only touches folders the user allowed.
Outgoing data	Requests to outside providers are scanned for secrets and personal data; the policy decides block, mask or allow.

Guard	What it does
Untrusted text	Text from documents, emails, web pages and tool results is information, never orders; a write action that follows such text always asks.
Answers	"Sources only" mode; a check for leaked secrets in replies.
Spend	Token and money limits per person, team and model.
Agent runs	The Companion's existing checks that stop stuck or looping runs.
Audit log	Detail
What is recorded	Who (person, API key or admin), when, from which device, the action, model or provider, documents used, tool calls and results, approvals, guard decisions, tokens and cost, sign-ins, and every admin change.
Content	Full, masked or none, as the retention policy says; records are deleted when their time is up.
Tamper evidence	Each record carries a fingerprint of the one before it, so an edited or deleted record shows. The server can only add records, never change them.
Use	A viewer with filters in the dashboard for the auditor role; export to the company's security monitoring system.

11 Ways to connect

Who	Address	Purpose
Apps	<code>/api/v1/...</code>	Chat, tasks, plugins, connections, collections
Admins	<code>/api/admin/v1/...</code>	Everything the dashboard does, so it can also be scripted
Other software	<code>/v1/chat/completions</code> , <code>/v1/models</code> , <code>/v1/embeddings</code>	OpenAI format: most AI tools work with a company key, still through the rules and the audit log
Desktop app	<code>/api/v1/devices/connect</code>	The device channel
Operations	<code>/healthz</code> , <code>/readyz</code> , <code>/metrics</code>	Health checks and measurements for the servers' monitoring
Anyone linking	<code>/api/openapi.json</code>	A machine-readable description of every endpoint

12 Installing and scaling

One machine (laptop or one server)

- One start script, including on a Windows laptop
- The server, PostgreSQL, and llama.cpp model processes, as the Companion runs them today
- No Redis; files in a folder
- Used for the pilot and for running models locally

Many servers (our own or AWS)

- Packaged as containers: sealed bundles that run the same on any server
- Run by Kubernetes, which starts copies across servers, replaces failed ones and adds or removes copies
- One install package (a Helm chart) for both our own servers and AWS

What scales	Signal	Notes
API servers	Requests and CPU load	Cheap to add; hold no state
Workers	Jobs waiting in the queue	Agent runs and document reading
Model pools	Requests waiting for that model (both engines report it)	Not CPU: a GPU server is full long before its CPU is busy

Limits worth knowing. On our own servers, scaling can only share out the GPUs we own between models; in AWS it can add GPU machines. A model can be set to switch off when idle, but the next request then waits for it to load, which can take minutes for a large model.

13 Technology choices

Area	Choice	Why
Backend	Rust (kept from the Companion)	Reuses the existing code; fast; same language as the desktop app's core
Database	PostgreSQL + pgvector	One database for records and document search
Between servers	Redis	Standard and simple; not needed on one machine
Files	S3, MinIO, or a folder	S3 in AWS, MinIO on our own servers
Model engines	llama.cpp and vLLM	Small machines and large GPU servers
Sign-in	OpenID Connect with Microsoft Entra ID	The standard way to use company accounts; other directories can be added later
MCP	The official Rust MCP library	Maintained with the standard itself
Desktop app	Tauri	Reuses the React screens; Rust core shares the tools code; small installers because it uses the browser engine built into the operating system
Admin dashboard	React	Same as today's screens; shared UI parts
Running servers	Containers, Kubernetes, Helm, KEDA	KEDA is the Kubernetes add-on that scales on signals like waiting requests
Monitoring	OpenTelemetry and Prometheus-format metrics	Works with the monitoring tools most companies already run

14 Action plan



Each phase ends with a review before the next one starts. The pilot (one department, its documents, 20–30 people) can start after phase 6 on one server. The design works across several servers from phase 1; phase 8 packages it and proves it under load.

Phase	Key work	Done when
0. Fork	Copy the Companion with its history (without the large model files), including its latest tool fixes; rename; all tests passing; this plan saved in the project.	The new project builds and every test passes.
1. Foundations	A map of every part of the current code and where it goes. PostgreSQL, with a one-off import of old history. Microsoft sign-in; users, groups, roles, API keys; an owner on every record. Settings with locks. Audit records from day one. The screens split into user app, admin dashboard and shared parts.	Two colleagues share one server and each sees only their own work; the admin locks a setting and the user sees it locked.
2. Models and gateway	Model list; several models at once; vLLM as a second engine; OpenAI, Anthropic and Google; the ability test for every model; same-conversation routing; the OpenAI-format address with keys; usage and limits.	Our models, Claude, GPT and Gemini in one list; a script with a company key works; limits stop overspending.

Phase	Key work	Done when
3. Desktop app	Tauri app; saved servers and connection test; sign-in through the system browser; tools moved into the shared library; device channel; pause and resume; signed installers and automatic updates.	Install the app, point it at the server, and the coding agent edits a local project.
4. Plugins and MCP	Plugin description format, catalogue, group access, user switches, ability check in the app and on the server, approval rules.	An admin adds an MCP server; only the chosen group sees it; a model that cannot use it shows it greyed out.
5. Jira and Outlook	Encrypted store for people's account connections; Jira Cloud; our Outlook connector; approval before sending or changing anything.	"Summarise my open tickets" and "draft a reply to this email" work; sending asks first.
6. Document search (RAG)	Collections, upload, reading in, search by meaning and keywords, access rights, citations, "sources only" mode, "never send outside" label.	The pilot can start on one server.
7. Guards and audit	Outgoing-data scan, untrusted-text rule, answer checks, tamper-evident chain, retention, export, viewer, auditor role.	An auditor can rebuild how any answer was produced.
8. Many servers	Containers, Kubernetes package, Redis between servers, scaling rules, GPU pools, health checks and metrics, load test, and the one-command Windows start.	Stopping a server mid-reply does not break a user's session; added load brings new copies up.
9. Hardening	Security review, a set of real test questions run on every change, guides for users and admins.	Ready for a wider rollout.

Time estimates come after phase 1's code map: it shows how much of today's request handling, which was written for one person, has to change.

15 What we need from the company

Item	Why	Needed by
A Microsoft Entra ID app registration, with admin consent	Company sign-in, and Outlook access on each person's behalf	Phase 1 (Outlook: phase 5)
Provider accounts and keys (OpenAI, Anthropic, Google)	Outside models; data-processing terms with each provider	Phase 2
A code-signing certificate	So Windows and macOS trust the app's installer and updates	Phase 3
Atlassian Cloud admin approval	Allows Atlassian's MCP server for our site	Phase 5
One GPU server (or AWS equivalent), a pilot department and its documents	The pilot	Phase 6
A legal review of the open-weight models' licences	Before the pilot uses them	Phase 6
The hosting decision: our own servers or AWS	Decides where the many-server version is installed	Phase 8

16 Risks and open points

Risk	How we handle it
The size of the rework	Today's request handling was written for one person and one model. Phase 1 starts with a full map of it before any estimate is given.

Risk	How we handle it
Hidden instructions in emails, documents and web pages	Such text is treated as information; actions that follow it, and every send or change, ask the user first.
Official connectors are limited	Microsoft's mail server needs a Copilot licence and Google's cannot send, so Outlook gets our own small connector.
Data leaving the company	Outside providers only through the outgoing-data scan, "never send outside" labels and per-group rules.
Speed with many copies of a model	The same conversation goes to the same copy, so it keeps reusing what it already read.
GPU capacity on our own servers	Fixed by the GPUs we buy; scaling shares them between models; AWS can add machines.
vLLM needs Linux GPU servers	Laptops and small servers use llama.cpp, which the Companion already tunes.

17 Terms used

Audit log	A record of who asked what, which documents and tools were used, and what was decided.
Container	A sealed bundle of a program and everything it needs, which runs the same on any server.
Embedding	A list of numbers that captures what a passage means, so passages can be found by meaning.
Kubernetes	The standard system for running containers across many servers; it restarts failed copies and adds or removes copies.
llama.cpp	A model server that runs open-weight models on almost any machine; the Companion uses it today.
Load balancer	The front door that spreads incoming requests across the server copies.
MCP	Model Context Protocol: an open standard for connecting AI apps to tools.
OpenAI format	The request format most AI tools speak; offering it lets existing tools use our models.
pgvector	An add-on that lets PostgreSQL search by meaning.
RAG	Retrieval-augmented generation: giving the model relevant passages from our documents when a question is asked.
Redis	Fast shared memory used so several servers can pass live replies and counters to each other.
Tauri	A way to package web screens as an installable desktop app with a small Rust core.
vLLM	A model server built to serve many people at once from large GPUs.